

OpenSeek-5 SemEval 2018 Task 1 Tweet Sadness Detection 技术报告

1. 项目概述

本项目面向 OpenSeek-5 任务，即 **SemEval 2018 Task 1 Tweet Sadness Detection**。任务目标是对输入推文进行二分类判断：识别推文作者是否表达了悲伤或广义负面情绪，并输出 `sad` 或 `not sad`。

与一般情感分类不同，本任务的难点不只是判断文本是否出现了 `sad`、`cry` 等显式词汇，而是需要结合推文中的短文本语境、表情符号、标签、幽默、讽刺、礼貌用语、成就语境等因素判断作者真实情绪。因此，本代码没有采用传统机器学习模型训练，也没有手写固定规则直接分类，而是采用 **大语言模型提示词推理 + 多提示词投票集成** 的方式完成预测。

当前实现的核心思路如下：

1. 从 JSON 数据集中读取任务 ID、训练样例和测试样例；
2. 对推文文本进行基础清洗，主要去除 `@username` 并规整空格；
3. 构建 6 个不同风格的情感判断提示词；
4. 对每条测试推文分别调用 Qwen3-4B 模型进行 6 次判断；
5. 将 6 次输出转换为 `sad` 或 `not sad`；
6. 使用多数投票得到最终预测；
7. 使用多线程并行处理测试集；
8. 将预测结果保存为 JSONL 格式，供平台提交。

与原有技术报告相比，本报告进一步补充了代码结构、模块职责、提示词设计逻辑、投票机制、并发推理、输出格式、可复现运行方式、代码中存在的问题以及后续优化方向。

2. 任务定义

2.1 任务名称

- 任务 ID: `openseek-5`
- 任务名称: `SemEval 2018 Task 1 Tweet Sadness Detection`
- 任务类型: 短文本情感二分类
- 输入: 一条英文推文文本
- 输出: `sad` 或 `not sad`

2.2 输入输出示例

输入示例：

```
1 | @user I am so upset 🙄
```

输出示例：

```
1 | sad
```

再如：

```
1 | Welcome and thanks for the greeting!
```

输出为：

```
1 | not sad
```

2.3 分类标准

本任务中的 `sad` 并不局限于狭义的“悲伤”。根据当前代码的提示词设计，`sad` 被扩展为一种广义负面情绪类别，包含以下情绪或表达：

- 悲伤、难过、哭泣、心碎；
- 失望、期待落空、不满；
- 沮丧、无力、痛苦；
- 愤怒、烦躁、厌恶；
- 明显抱怨或强烈负面评价；
- 第一人称表达出的负面状态；
- 明确的负面情绪标签。

而 `not sad` 不仅包括积极文本，也包括以下容易误判的情况：

- 纯积极、祝福、感谢、欢迎类表达；
- 礼貌性的 `sorry`；
- 成就完成后的哭泣表情；
- 纯客观事实陈述；
- 纯幽默或夸张表达；
- 宗教引用或固定文本；
- 只有标签但正文无真实情绪；
- 仅表达身体疲劳但无明显负面情绪。

3. 数据集结构

代码默认从如下 JSON 文件读取数据：

```
1 | file_path = r"D:\WorkSpace\python\flagOS赛题三\LongContext-ICL-Annotation\data\openseek-5_semeval_2018_task1_tweet_sadness_detection.json"
```

数据加载函数为：

```
1 def task5_data_loader(file_path):
2     with open(file_path, 'r', encoding='utf-8') as f:
3         data = json.load(f)
4         return data.get("task_id"), data.get("examples", []), data.get("test_samples", [])
```

该函数返回三个对象：

字段	含义
task_id	当前任务编号，例如 <code>openseek-5</code>
examples	带标签训练样例，可用于提示词设计、调试或本地验证
test_samples	无标签测试样例，需要生成最终预测

每条样例一般具有如下结构：

```
1 {
2     "id": "openseek-5-xxxx",
3     "input": "tweet text here",
4     "output": ["sad"]
5 }
```

测试集样例通常不包含 `output` 字段：

```
1 {
2     "id": "openseek-5-xxxx",
3     "input": "tweet text here"
4 }
```

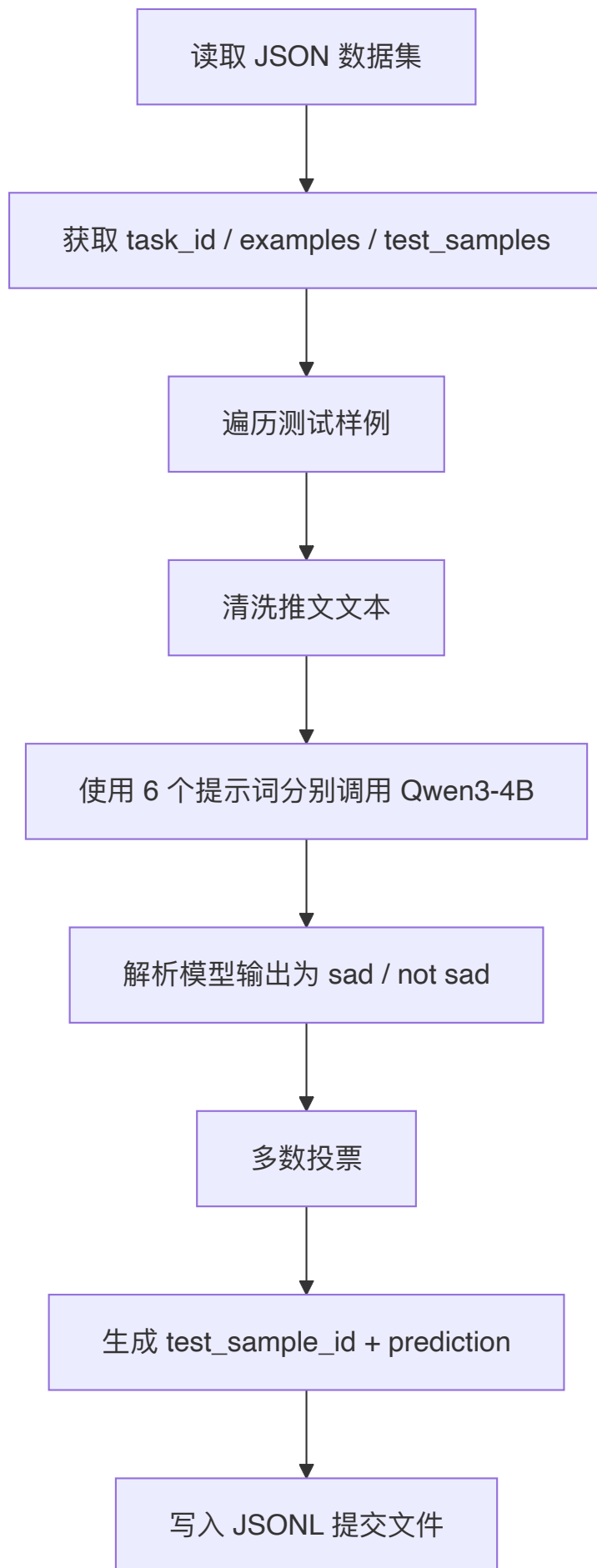
当前代码虽然加载了 `examples`，但主流程中实际只使用了 `test_samples` 进行预测，未对训练样例进行本地准确率验证。

4. 方法总体设计

4.1 总体技术路线

本任务采用的是 **Prompt Ensemble Classification**，即提示词集成分类方法。其本质是将一个二分类问题转化为多次大语言模型判断，再通过投票融合多个判断结果。

整体流程如下：



4.2 为什么使用多提示词投票

推文情感判断存在较多边界情况。例如：

- 同样出现哭泣表情，可能是真悲伤，也可能是激动或夸张；
- `sorry` 可能是真正道歉或悲伤，也可能只是礼貌用语；
- `rage`、`frustrated` 等词虽然不是 `sadness` 的狭义词，但在该任务中可能仍属于负面情绪；
- `lol`、`haha` 可能代表幽默，但也可能和真实负面情绪混合出现；
- `hashtag` 可能是情绪标签，也可能只是装饰性标签。

单一提示词容易对某类模式过度敏感，从而产生误报或漏报。因此代码设计了 6 个不同角度的提示词，包括精准规则版、few-shot 示例版、严格规则版、分步判断版、权重评分版和综合版。多个提示词的判断结果再通过投票融合，以降低单个提示词不稳定带来的风险。

5. 代码模块详细分析

5.1 依赖导入

代码导入的主要模块如下：

```
1 import json
2 import os
3 import time
4 from openai import OpenAI
5 from tqdm import tqdm
6 from concurrent.futures import ThreadPoolExecutor
7 from functools import partial
8 import re
```

各依赖作用如下：

依赖	作用
<code>json</code>	读取任务数据集，写入 JSONL 输出
<code>os</code>	创建输出目录
<code>time</code>	API 重试等待时使用
<code>OpenAI</code>	使用 OpenAI 兼容接口调用 Qwen3-4B
<code>tqdm</code>	显示测试集处理进度条
<code>ThreadPoolExecutor</code>	并发处理多条测试样例
<code>partial</code>	固定函数部分参数，便于并发 map 调用
<code>re</code>	使用正则表达式清洗推文文本

5.2 数据加载模块

数据加载函数如下：

```
1 def task5_data_loader(file_path):
2     with open(file_path, 'r', encoding='utf-8') as f:
3         data = json.load(f)
4         return data.get("task_id"), data.get("examples", []), data.get("test_samples", [])
```

该函数实现较简单，主要完成三项工作：

1. 使用 UTF-8 编码读取 JSON 文件；
2. 从 JSON 顶层对象中获取 `task_id`；
3. 获取训练样例 `examples` 和测试样例 `test_samples`。

使用 `data.get(..., [])` 可以避免字段缺失时直接报错。如果数据中没有 `examples` 或 `test_samples` 字段，函数会返回空列表，从而提高程序鲁棒性。

不过，该函数目前没有读取 `task_name` 和 `Definition` 字段。如果后续希望自动根据任务说明生成提示词，或者在报告中自动记录完整任务信息，可以扩展为：

```
1 return (
2     data.get("task_id", ""),
3     data.get("task_name", ""),
4     data.get("Definition", []),
5     data.get("examples", []),
6     data.get("test_samples", [])
7 )
```

5.3 模型接口配置

当前代码使用 OpenAI 兼容接口访问 Qwen3-4B：

```
1 client = OpenAI(
2     api_key="dummy",
3     base_url="https://flagos.io/flagos-lab/hw/node/HW-gpu57/port/22653/v1"
4 )
```

模型名称为：

```
1 /Qwen3-4B/Qwen/Qwen3-4B
```

该设置说明当前环境中的模型服务提供了类 OpenAI Chat Completions API，因此可以直接通过：

```
1 client.chat.completions.create(...)
```

进行调用。

需要注意的是，`api_key="dummy"` 表示接口可能不校验密钥，或者实际鉴权由平台环境完成。为了增强代码可移植性，建议后续改为环境变量方式：

```
1 client = OpenAI(  
2     api_key=os.getenv("OPENAI_API_KEY", "dummy"),  
3     base_url=os.getenv("OPENAI_BASE_URL")  
4 )
```

这样可以避免将服务地址、密钥等配置硬编码到代码中。

5.4 API 重试函数

代码中定义了 `qwen_api`：

```
1 def qwen_api(messages, model="/Qwen3-4B/Qwen/Qwen3-4B", retries=3):  
2     for attempt in range(retries):  
3         try:  
4             res = client.chat.completions.create(  
5                 model=model,  
6                 messages=messages,  
7                 temperature=0.0,  
8             )  
9             return res.choices[0].message.content  
10        except Exception as e:  
11            if attempt == retries - 1:  
12                print(f"\nAPI 调用失败: {e}")  
13                return ""  
14            time.sleep(2)
```

该函数具有以下特点：

1. 最多重试 3 次；
2. 每次失败后等待 2 秒；
3. 最后一次仍失败则返回空字符串；
4. 使用 `temperature=0.0` 保证输出尽可能稳定。

但是需要特别指出：当前主预测流程并没有实际调用 `qwen_api`。真正用于预测的是 `get_prediction()`，它直接调用了 `client.chat.completions.create()`，而没有复用这里的重试机制。因此，虽然代码中存在 API 重试函数，但实际测试集预测过程并没有获得该函数提供的重试保护。

这属于当前代码的一个重要可改进点。更合理的做法是让 `get_prediction()` 调用 `qwen_api()`，统一模型调用逻辑。

5.5 文本清洗模块

当前代码定义了如下清洗函数：

```

1 def clean_text(text):
2     cleaned_text = re.sub(r'@\w+', '', text)
3     cleaned_text = re.sub(r'\s+', ' ', cleaned_text).strip()
4     return cleaned_text

```

该函数主要完成两步：

1. 删除推文中的用户名引用，例如 @user、@abc123；
2. 将连续空格、换行符、制表符等规整为单个空格。

清洗前：

```

1 @user I am so upset today

```

清洗后：

```

1 I am so upset today

```

这样处理的原因是推文中的用户名通常不包含情感信息，保留它可能干扰模型判断。例如，模型可能错误关注用户名而忽略正文。

不过，当前清洗逻辑仍然较保守，没有删除 URL、HTML 实体、重复标点等内容。对于推文任务来说，可以考虑进一步扩展：

```

1 text = re.sub(r'https?://\S+|www\.\S+', '', text)
2 text = re.sub(r'&','&', text)
3 text = re.sub(r'\s+', ' ', text).strip()

```

但需要注意，不能盲目删除 hashtag 和 emoji，因为它们在本任务中可能包含重要情绪信号。

5.6 提示词集成模块

当前代码的核心是 PROMPTS 列表，其中包含 6 个提示词模板。每个模板都包含 {text} 占位符，用于插入当前推文：

```

1 content = prompt.format(text=text)

```

6 个提示词的设计目标并不完全相同，具体如下。

5.6.1 Prompt 1：精准规则版

Prompt 1 以规则为主，结构包括：

- 宽泛悲伤定义；
- 立即排除规则；
- 特别注意的 not sad 情况；

- 确认 sad 的条件；
- 特殊判断规则；
- 严格要求只输出 `sad` 或 `not sad`。

该提示词的优点是覆盖全面，对容易误判的情况进行了细粒度约束，例如礼貌用语、成就场景、宗教引用、纯标签、疲劳表达等。

5.6.2 Prompt 2: Few-shot 示例版

Prompt 2 在规则基础上加入了示例学习，列举了多个典型输入和对应判断。其作用是让模型通过具体样例理解边界情况，例如：

- 成就语境中的哭泣表情不一定是悲伤；
- 礼貌性的 `sorry` 不应直接判断为悲伤；
- 愤怒、失望、强烈厌恶可归入广义负面情绪；
- 纯幽默或夸张表达需要排除。

Few-shot 提示词通常适合处理语义边界模糊的问题，因为它不仅告诉模型规则，还提供了可模仿的判断模式。

5.6.3 Prompt 3: 严格规则版

Prompt 3 更强调“先排除，再确认”的判断方式。它把 `not sad` 的排除条件放在前面，要求模型优先识别纯积极、礼貌、幽默、宗教引用等非悲伤文本。

这种设计有利于降低误报，即避免模型因为看到某个单独负面词或表情符号就直接输出 `sad`。

5.6.4 Prompt 4: 分步判断版

Prompt 4 将判断过程拆分为 16 个步骤。它的逻辑类似人工标注流程：

1. 先判断是否纯积极；
2. 再判断是否成就场景；
3. 再判断是否礼貌用语；
4. 继续判断幽默、宗教、标签、疲劳等排除项；
5. 最后识别悲伤、愤怒、烦躁、失望、痛苦等负面情绪。

分步提示词的优点是显式引导模型进行层级判断，减少一步到位分类时的随机性。

5.6.5 Prompt 5: 权重评分版

Prompt 5 采用评分思想，把不同情绪信号赋予正负权重。例如：

- 强烈负面情绪给更高正分；
- 明显负面情绪给中等正分；
- 成就场景、礼貌用语、纯积极、宗教引用等给负分；
- 最后根据总分判断 `sad` 或 `not sad`。

虽然模型并不一定真的进行严格数学加减，但这种提示方式可以帮助模型理解不同证据的强弱关系。

5.6.6 Prompt 6：综合版

Prompt 6 是综合规则版，整合了前面几个提示词中的核心规则，强调先排除非悲伤语境，再识别真实负面情绪。

该提示词在投票中起到“稳定器”作用，避免过度依赖某一种提示词风格。

5.7 系统提示词设计

代码中定义了全局系统提示词：

```
1 SYSTEM_PROMPT = "你是一位情感分类专家，请根据提示词判断文本是否表达负面情绪。只输出 sad 或 not sad。"
```

该系统提示词主要起到两个作用：

1. 设定模型角色为情感分类专家；
2. 强制输出格式只能是 sad 或 not sad。

主流程中另外定义了一个局部变量：

```
1 system_prompt = "你是情感分析专家，能识别文字背后的真实情绪。特别注意讽刺、幽默、表情符号的语境含义。"
```

并通过 `partial` 传入 `process_single_sample()`。但是当前 `process_single_sample()` 函数内部并没有使用这个参数，`get_prediction()` 仍然使用全局 `SYSTEM_PROMPT`。因此，主流程中的 `system_prompt` 实际上没有生效。

这说明代码中存在一定冗余。如果希望该局部系统提示词生效，需要修改函数调用链，让 `get_prediction()` 接收并使用 `system_prompt` 参数。

5.8 单次预测函数

核心预测函数如下：

```
1 def get_prediction(text, prompt, temperature):
2     content = prompt.format(text=text)
3     messages = [
4         {"role": "system", "content": SYSTEM_PROMPT},
5         {"role": "user", "content": content}
6     ]
7
8     try:
9         res = client.chat.completions.create(
10             model="/Qwen3-4B/Qwen/Qwen3-4B",
11             messages=messages,
```

```

12         temperature=temperature
13     )
14     output = res.choices[0].message.content.lower().strip()
15     if 'sad' in output and 'not sad' not in output:
16         return 'sad'
17     return 'not sad'
18 except:
19     return None

```

该函数的流程如下：

1. 将清洗后的文本插入提示词模板；
2. 构建 Chat Completions API 所需的 messages；
3. 调用 Qwen3-4B 模型；
4. 将输出转为小写并去除首尾空格；
5. 如果输出包含 `sad` 且不包含 `not sad`，则判为 `sad`；
6. 否则统一判为 `not sad`；
7. 如果 API 调用异常，则返回 `None`。

这里的输出解析逻辑较简洁，能够处理模型输出中包含额外解释的情况。例如：

```
1 | sad
```

或：

```
1 | The answer is sad.
```

都会被解析为 `sad`。

但该逻辑也有一个特点：只要输出中含有 `not sad`，即使同时出现 `sad`，最终也会返回 `not sad`。这是为了避免 `not sad` 中包含子串 `sad` 导致误判。

5.9 单样例处理与投票机制

单条样例处理函数如下：

```

1  def process_single_sample(sample, system_prompt, task_id):
2      sample_id = sample.get("id", "")
3      input_text = clean_text(sample["input"])
4      votes = {'sad': 0, 'not sad': 0}
5      predictions = []
6      n_models = len(PROMPTS)
7      for i in range(min(n_models, len(PROMPTS))):
8          pred = get_prediction(input_text, PROMPTS[i], 0.0)
9          if pred:
10             votes[pred] += 1
11             predictions.append(pred)

```

```

12
13     if votes['sad'] > votes['not sad']:
14         prediction = 'sad'
15     else:
16         prediction = 'not sad'
17
18     return {
19         "task_id": task_id,
20         "sample_id": sample_id,
21         "input": input_text,
22         "model_output": prediction,
23     }

```

该函数对每条推文执行完整预测流程：

1. 读取样例 ID；
2. 清洗输入文本；
3. 初始化投票计数器；
4. 遍历 6 个提示词；
5. 每个提示词调用一次模型；
6. 记录 `sad` 和 `not sad` 的票数；
7. 如果 `sad` 票数严格大于 `not sad`，输出 `sad`；
8. 否则输出 `not sad`。

投票规则可以写成：

```

1 prediction = 'sad' if votes['sad'] > votes['not sad'] else 'not sad'

```

这意味着：

- 4:2、5:1、6:0 时输出 `sad`；
- 3:3 平票时输出 `not sad`；
- 如果部分 API 调用失败导致有效票数减少，也按当前有效票数判断；
- 如果所有调用都失败，`sad=0` 且 `not sad=0`，最终也会输出 `not sad`。

这种设计偏保守，可以减少误报，但也可能增加漏报。对于悲伤检测任务，如果平台更看重召回率，可以考虑把平票改为 `sad`，或者对 Prompt 2 / Prompt 4 等更可靠提示词设置更高权重。

5.10 并发推理模块

主流程使用 `ThreadPoolExecutor` 并发处理测试样例：

```

1 process_func = partial(process_single_sample, system_prompt=system_prompt,
2 task_id=task5_id)
3
4 max_workers = 200
5 results_list = []
6
7 with ThreadPoolExecutor(max_workers=max_workers) as executor:
8     for result in tqdm(executor.map(process_func, eval_data), total=total_cnt,
9 desc="Evaluating"):
10         results_list.append(result)

```

这里使用 `executor.map()` 而不是 `as_completed()`，其主要好处是：

- 并发执行；
- 返回结果顺序与输入顺序一致；
- 方便后续直接生成提交文件；
- 不需要手动维护样例索引。

每条样例会调用 6 次模型，因此如果测试集数量为 `N`，总 API 调用次数约为：

```
1 6 × N
```

例如：

- 100 条测试样例：约 600 次模型调用；
- 1000 条测试样例：约 6000 次模型调用。

当前 `max_workers=200`，并发度非常高。它可以显著提高速度，但也可能带来以下风险：

1. 模型服务限流；
2. 网络连接过多导致请求失败；
3. 服务器压力过大；
4. 日志难以排查；
5. API 异常时大量样例回退为 `not sad`。

更稳妥的设置通常是先从 `16`、`32` 或 `64` 开始，根据模型服务吞吐能力逐步调高。

5.11 输出文件生成

模型推理完成后，代码将结果整理为提交格式：

```
1 output_data = [  
2     {  
3         "test_sample_id": item["sample_id"],  
4         "prediction": item["model_output"]  
5     }  
6     for item in results_list  
7 ]
```

最终写入 JSONL 文件：

```
1 os.makedirs(os.path.dirname(out_path), exist_ok=True)  
2  
3 with open(out_path, 'w', encoding='utf-8') as f:  
4     for item in output_data:  
5         line = json.dumps(item, ensure_ascii=False)  
6         f.write(line + '\n')
```

输出路径为：

```
1 out_path = r"D:\Workspace\python\flag05赛题三\LongContext-ICL-  
Annotation\rgs_q5\experiment\openseek-5-v1.jsonl"
```

输出文件每一行是一个 JSON 对象，例如：

```
1 {"test_sample_id": "openseek-5-f296990fa391466eb3196f899d2a7b4e", "prediction": "not  
sad"}
```

该格式适合比赛平台逐行读取和评测。

6. 提示词策略分析

6.1 广义负面情绪定义

原任务名称是 Tweet Sadness Detection，看似只判断悲伤，但从当前代码设计可以看出，作者将 `sad` 扩展成广义负面情绪类别。这样做的原因是，任务样例中可能并不只包含直接悲伤表达，还可能包含：

- 愤怒；
- 失望；
- 痛苦；
- 烦躁；
- 厌恶；
- 抱怨；
- 心理低落；
- 明显负面情绪标签。

如果只识别狭义 sadness，可能漏掉大量真实标注为 `sad` 的样例。因此提示词中多次强调“宽泛悲伤定义”。

6.2 排除规则的重要性

推文数据的一大难点是表达方式非常非正式。例如，哭泣表情不一定代表悲伤，`sorry` 不一定代表负面情绪，`lol` 也不一定代表快乐。为了减少误报，提示词加入了大量排除规则，包括：

- 纯积极表达；
- 成就完成语境；
- 礼貌用语；
- 纯中性客观信息；
- 纯幽默文本；
- 祝福和感谢；
- 宗教引用；
- 讽刺或夸张表达；
- 纯标签；
- 单纯疲劳。

这些排除规则体现了本任务的核心难点：不能只依赖单个关键词，而要结合语境判断作者真实情绪。

6.3 确认规则的作用

在排除非悲伤语境后，提示词再通过确认规则判断 `sad`。确认规则主要覆盖以下信号：

- 明确悲伤词；
- 愤怒、厌恶、强烈负面评价；
- 烦躁和挫败；
- 失望和不满；
- 痛苦和受伤；
- 抱怨句式；
- 第一人称负面情绪表达；
- 明确情绪标签。

这种“先排除、再确认”的方式比单纯枚举 sad 词表更稳健。

6.4 多提示词互补性

6 个提示词并不是简单重复，而是从不同方向增强判断稳定性：

提示词	主要特点	作用
Prompt 1	精准规则版	覆盖完整规则，降低漏判
Prompt 2	Few-shot 示例版	用示例校准边界情况
Prompt 3	严格规则版	强调先排除非悲伤语境
Prompt 4	分步判断版	引导模型按流程推理
Prompt 5	权重评分版	显式表达不同证据强弱
Prompt 6	综合版	平衡规则覆盖与输出稳定性

通过投票融合后，单个提示词偶然误判对最终结果的影响会降低。

7. 实验流程说明

7.1 当前代码实际实验流程

当前主函数执行流程如下：

```
1 task5_id, examples, test_samples = task5_data_loader(file_path)
2 eval_data = test_samples
```

也就是说，代码将测试集作为预测对象，不使用训练集进行本地验证。

随后：

- 1. 使用 `partial()` 固定 `system_prompt` 和 `task_id`；
- 2. 使用 `ThreadPoolExecutor(max_workers=200)` 并发处理所有测试样例；
- 3. 每条测试样例调用 6 个提示词；
- 4. 对 6 个结果进行投票；
- 5. 将最终结果写为 JSONL。

7.2 当前代码没有执行训练集准确率评估

虽然数据加载函数返回了 `examples`，但当前主流程没有使用它们计算准确率。因此代码不能直接输出本地 `accuracy`。

如果需要在训练样例或开发样例上验证效果，可以加入如下逻辑：

```
1 eval_data = examples
2 correct = 0
3 for item in results_list:
4     # 需要保留原始 output 标签后再比较
5     pass
```


或者写一个单独的评估函数：

```
1 def evaluate_examples(examples, task_id):
2     correct = 0
3     for sample in examples:
4         result = process_single_sample(sample, system_prompt="", task_id=task_id)
5         gt = sample["output"][0].lower().strip()
6         pred = result["model_output"]
7         if pred == gt:
8             correct += 1
9     return correct / len(examples) if examples else 0.0
```

这样可以更清楚地比较不同提示词版本、不同投票策略和不同并发设置的实际效果。

7.3 输出结果

当前程序最终输出：

```
1 评测结果已成功保存为 JSONL 格式：D:\WorkSpace\python\flagOS赛题三\LongContext-ICL-
  Annotation\rgs_q5\experiment/openseek-5-v1.jsonl
```

输出文件格式示例：

```
1  {"test_sample_id": "openseek-5-f296990fa391466eb3196f899d2a7b4e", "prediction": "not
  sad"}
```